Building U-Boot for the BeagleBone AI-64

SoC: TI J721E (TDA4VM) · U-Boot: **v2026.01-Beagle** · Source: `~/Courses/BBB/build/u-boot`

> **Verified.** Every command in this document was executed end-to-end on Ubuntu 24.04 (Python 3.12) against the `v2026.01-Beagle` tree. Both the R5 and A72 builds complete with exit status 0, and `tispl.bin_unsigned` was confirmed to contain TF-A, OP-TEE, the DM binary, the 64-bit SPL and the `k3-j721e-beagleboneai64` device tree.

1. What you are actually building

The J721E is a multi-core SoC and uses a **split-binary boot flow**. A single `make` is not enough: you compile U-Boot **twice** (once for the Cortex-R5 boot processor, once for the Cortex-A72 application processor), and you must supply three external components that get packaged into the final images.

Image	Runs on	Produced by
`tiboot3.bin`	Cortex-R5 (wakeup domain)	U-Boot R5 build
`sysfw.itb`	Security enclave (TIFS/SYSFW)	U-Boot R5 build
`tispl.bin`	Cortex-A72 (main domain)	U-Boot A72 build (packs TF-A + OP-TEE + DM)
`u-boot.img`	Cortex-A72	U-Boot A72 build

> **Why `sysfw.itb` exists here.** J721E and AM65x use the split flow, where TIFS and DM firmware ship separately. Newer SoCs (AM62x, AM64x) fold everything into `tiboot3.bin` . Do not follow an AM62x guide for this board — you will be missing a file.

2. Prerequisites

2.1 Cross compilers

You need **both** a 32-bit and a 64-bit ARM toolchain — the R5 is ARMv7, the A72 is ARMv8.

```
sudo apt update
sudo apt install gcc-arm-linux-gnueabihf gcc-aarch64-linux-gnu
```

2.2 Host build dependencies

U-Boot's `binman` packaging tool needs Python modules, and the FIT image tooling needs OpenSSL and device-tree headers. Install everything from apt in one shot:

```
sudo apt update
sudo apt install build-essential bison flex libssl-dev libgnutls28-dev \
    device-tree-compiler swig python3-dev python3-setuptools \
    python3-pyelftools python3-yaml python3-jsonschema yamllint \
    uuid-dev libyaml-dev
```

Verify the Python side is importable by the *system* interpreter, which is what binman runs under:

```
python3 -c "import elftools, yaml, jsonschema, yamllint; print('binman deps ok')"
```

The package is `yamllint`, not `python3-yamllint`. There is no `python3-yamllint` in the Ubuntu archive — the `yamllint` package installs the importable module. It is easy to miss because it is not needed by generic U-Boot builds: only the K3-specific `ti-board-config` binman entry type imports it. Omit it and the build runs for several minutes, compiles everything successfully, and dies at the very last step with `Unknown entry type 'ti-board-config'`.

Ubuntu ships yamllint 1.33.0 while `tools/binman/requirements.txt` pins 1.35.1. The pin exists for U-Boot's CI; binman only needs the import to succeed, and 1.33.0 works.

Do not use `pip3 install` here. On Ubuntu 24.04 / Debian 12 and newer, the system Python is marked *externally managed* (PEP 668) and `pip3 install pyelftools` fails with `error: externally-managed-environment`. Install from apt as shown above. Resist the `--break-system-packages` override that pip suggests: it clobbers apt-managed files to install packages Debian already ships. A virtualenv also works, but it would have to be activated for every `make` invocation, since binman is invoked by the U-Boot Makefile using the system `python3`.

Most common build failure. A missing `pyelftools` or `swig` makes the build die deep inside binman with an obscure traceback, long after U-Boot itself has compiled fine. Install these *before* you start.

2.3 Source repositories

Clone the three companions next to your existing `u-boot` directory.

```
cd ~/Courses/BBB/build

git clone https://github.com/TexasInstruments/ti-linux-firmware.git \
    -b ti-linux-firmware

git clone https://git.trustedfirmware.org/TF-A/trusted-firmware-a.git

git clone https://github.com/OP-TEE/optee_os.git
```

Version matching matters. `ti-linux-firmware` is the one to be careful with — the DM and TIFS binaries it carries must be new enough for a 2026.01 U-Boot. If the board hangs before the A72 console appears, a stale firmware checkout is the first thing to suspect.

2.4 Preflight: check the U-Boot clone is intact

Do this **before** you build. If the `u-boot` tree was ever cloned on Windows, or checked out onto an NTFS/exFAT volume and then copied to Linux, git will have silently written every symlink as an ordinary text file and dropped every executable bit. The build gets most of the way through and then fails in confusing ways.

```
cd $UBOOT_PATH

# Both must print 'true'. If either prints 'false', the clone is damaged.
git config core.symlinks
git config core.fileMode

# binman must be a symlink, not a text file:
file tools/binman/binman
# GOOD: tools/binman/binman: symbolic link to main.py
# BAD : tools/binman/binman: ASCII text, with no line terminators
```

Repairing a damaged clone

Re-cloning on the Linux filesystem is the cleanest fix. If you must repair in place, do this from a **clean** working tree (`git status` shows no modifications — otherwise you will lose work):

```
cd $UBOOT_PATH
git config core.symlinks true
git config core.fileMode true

# Recreate every symlink from the git index
git ls-files -s | awk '$1=="120000"{sub(/^[^\t]*\t/,""); printf "%s%c", $0, 0}' \
    | git checkout-index -f -z --stdin

# Restore every executable bit from the git index
git ls-files -s | awk '$1=="100755"{sub(/^[^\t]*\t/,""); print}' \
    | tr '\n' '\0' | xargs -0 chmod +x

# Confirm: both counts must be 0
git ls-files -s | awk '$1=="120000"{sub(/^[^\t]*\t/,""); print}' \
    | while read -r f; do [ -L "$f" ] || echo "broken: $f"; done | wc -l
git ls-files -s | awk '$1=="100755"{sub(/^[^\t]*\t/,""); print}' \
    | while read -r f; do [ -x "$f" ] || echo "non-exec: $f"; done | wc -l
```

Do not simply `chmod +x tools/binman/binman`. On a damaged clone that file is not a program — it is a 7-byte text file containing the string `main.py`. Marking it executable produces executable garbage, not a working binman. The symlink has to be recreated, which is what the `git checkout-index` line above does.

After setting `core.fileMode=true`, `git status` may report a few `.bat` files under `lib/lwip` and `lib/mbedtls` as modified. They were already mode 755 from the bad copy; git had merely been ignoring modes until now. They are Windows batch scripts, irrelevant to the build. Restore them with `chmod 644 <file>` if you want a clean `git status`.

3. Set up the environment

Everything below assumes these variables. Set them once per shell session — if you open a new terminal, set them again.

```
# --- Toolchains ---
export CC32=arm-linux-gnueabihf-
export CC64=aarch64-linux-gnu-

# --- Source paths (adjust the base if yours differs) ---
export BASE=~/Courses/BBB/build
export UBOOT_PATH=$BASE/u-boot
export LNX_FW_PATH=$BASE/ti-linux-firmware
export TFA_PATH=$BASE/trusted-firmware-a
export OPTEE_PATH=$BASE/optee_os

# --- BeagleBone AI-64 specific ---
export UBOOT_CFG_CORTEXR=j721e_beagleboneai64_r5_defconfig
export UBOOT_CFG_CORTEXA=j721e_beagleboneai64_a72_defconfig
export TFA_BOARD=generic
export OPTEE_PLATFORM=k3-j721e
```

Confirm the defconfigs really exist in your tree before going further:

```
ls $UBOOT_PATH/configs/ | grep beagleboneai64
# expected:
#   j721e_beagleboneai64_a72_defconfig
#   j721e_beagleboneai64_r5_defconfig
```

4. Build step 1 — Trusted Firmware-A (TF-A)

This is the first thing that runs on the A72 cores. `SPD=opteed` wires in the OP-TEE dispatcher.

```
cd $TFA_PATH

make CROSS_COMPILE=$CC64 ARCH=aarch64 PLAT=k3 \
     TARGET_BOARD=$TFA_BOARD SPD=opteed
```

Output: `$TFA_PATH/build/k3/generic/release/bl31.bin`

5. Build step 2 — OP-TEE OS

OP-TEE is the secure-world companion to the Linux kernel. It needs *both* compilers.

```
cd $OPTEE_PATH

make CROSS_COMPILE=$CC32 CROSS_COMPILE64=$CC64 \
     CFG_ARM64_core=y PLATFORM=$OPTEE_PLATFORM
```

Output: `$OPTEE_PATH/out/arm-plat-k3/core/tee-raw.bin`

Verify both outputs exist before moving on. The U-Boot A72 build references them by absolute path, and binman will happily produce a broken image rather than error loudly if a path is wrong.

```
ls -l $TFA_PATH/build/k3/$TFA_BOARD/release/bl31.bin
ls -l $OPTEE_PATH/out/arm-plat-k3/core/tee-raw.bin
```

6. Build step 3 — U-Boot for the Cortex-R5

This produces the wakeup-domain bootloader and the system firmware image. Build into a separate output directory (`O=`) so the R5 and A72 builds never collide.

```
cd $UBOOT_PATH

make O=build/r5 $UBOOT_CFG_CORTEXR

make O=build/r5 CROSS_COMPILE=$CC32 BINMAN_INDIRS=$LNX_FW_PATH
```

Outputs in `$UBOOT_PATH/build/r5/` :

```
tiboot3-j721e-gp-evm.bin        <- the real image
tiboot3.bin -> ./tiboot3-j721e-gp-evm.bin   <- symlink, made by binman

sysfw-j721e-gp-evm.itb          <- the real image
sysfw.itb -> ./sysfw-j721e-gp-evm.itb       <- symlink, made by binman
```

> **binman names them for you.** It emits `tiboot3.bin` and `sysfw.itb` as symlinks pointing at the correct variant, so no manual renaming is needed — just copy them (`cp` follows symlinks by default). For the BeagleBone AI-64 defconfig, only the **GP** variants are built; you will not see `-hs-fs-` or `-hs-` files, because the board ships with GP (general purpose) TDA4VM silicon. If you ever build for an EVM or SK board, several variants appear and the symlink tells you which one is current.

7. Build step 4 — U-Boot for the Cortex-A72

Now feed TF-A and OP-TEE into the A72 build so binman can pack `tispl.bin` .

```
cd $UBOOT_PATH

make O=build/a72 $UBOOT_CFG_CORTEXA

make O=build/a72 CROSS_COMPILE=$CC64 \
     BINMAN_INDIRS=$LNX_FW_PATH \
     BL31=$TFA_PATH/build/k3/$TFA_BOARD/release/bl31.bin \
     TEE=$OPTEE_PATH/out/arm-plat-k3/core/tee-raw.bin
```

Outputs in `$UBOOT_PATH/build/a72/` :

- `tispl.bin_unsigned` ← use this on GP silicon
- `u-boot.img_unsigned` ← use this on GP silicon

> **There is no plain `tispl.bin` here.** Unlike the R5 step, binman does not create a convenience symlink for it. On GP silicon you take the `_unsigned` files and rename them during the copy (next section). A `u-boot.img` without the suffix also exists — it is the signed-flow artifact and is *not* the one you want.

Confirm the packaging actually worked before you touch an SD card:

```
cd $UBOOT_PATH/build/a72
./tools/dumpimage -l tispl.bin_unsigned | grep -iE "description|type"
```

You should see all five components — if TF-A or OP-TEE is missing, your `BL31=` or `TEE=` path was wrong:

```
FIT description: Configuration to load ATF and SPL
  Description:  ARM Trusted Firmware
  Description:  OP-TEE
  Description:  DM binary
  Description:  SPL (64-bit)
  Description:  k3-j721e-beagleboneai64
```

> The DM firmware is pulled automatically from `BINMAN_INDIRS`. You only need to pass an explicit `TI_DM=<path>` if you are substituting a custom DM binary.

8. Assemble the boot files

The ROM and the R5 SPL look for **exact filenames**. The two R5 images already carry the right names (via binman's symlinks, which `cp` follows); the two A72 images must be renamed as you copy.

```
cd $UBOOT_PATH
mkdir -p $BASE/deploy

cp build/r5/tiboot3.bin           $BASE/deploy/tiboot3.bin
cp build/r5/sysfw.itb             $BASE/deploy/sysfw.itb
cp build/a72/tispl.bin_unsigned   $BASE/deploy/tispl.bin
cp build/a72/u-boot.img_unsigned  $BASE/deploy/u-boot.img

ls -l $BASE/deploy/
```

All four must be present, and none may be a dangling symlink:

```
tiboot3.bin    ~207 KiB
sysfw.itb      ~263 KiB
tispl.bin      ~998 KiB
u-boot.img     ~1.2 MiB
```

9. Deploy to the SD card

The AI-64's boot partition is a FAT partition flagged bootable. Mount it and copy all four files into the root of that partition.

```
# Identify the card first — do NOT guess the device node.
lsblk

# Assuming the FAT boot partition is /dev/sdX1 :
sudo mount /dev/sdX1 /mnt

sudo cp $BASE/deploy/tiboot3.bin /mnt/
sudo cp $BASE/deploy/tispl.bin   /mnt/
sudo cp $BASE/deploy/u-boot.img  /mnt/
sudo cp $BASE/deploy/sysfw.itb   /mnt/

sync
sudo umount /mnt
```

Check the device node. `/dev/sdX` is a placeholder. Writing to the wrong disk will destroy your host filesystem. Run `lsblk` with the card unplugged, then again plugged in, and use the node that appeared.

10. Verify on the board

Connect to the debug UART (3.3 V, **115200 8N1**), insert the card, and power on. A correct boot shows the R5 SPL banner first, then the A72 SPL, then the U-Boot prompt:

```
U-Boot SPL 2026.01 ...    <- R5, from tiboot3.bin
...
U-Boot SPL 2026.01 ...    <- A72, from tispl.bin
...
U-Boot 2026.01 ...
=>
```

11. Rebuilding cleanly

If you change defconfigs or switch branches, wipe the output directories rather than trusting incremental state:

```
cd $UBOOT_PATH
rm -rf build/r5 build/a72
```

Then repeat sections 6 and 7. TF-A and OP-TEE only need rebuilding if you change *them*.

12. Troubleshooting quick reference

Symptom	Likely cause
`scripts/gcc-version.sh: Permission denied` followed by `Kconfig:91: syntax error`	Exec bits stripped from the clone. Repair per §2.4 — do not chmod file-by-file, there are 189 of them
`tools/binman/binman: Permission denied` at the `BINMAN .binman_stamp` step	Symlinks materialized as text files. `binman` is a 7-byte file reading `main.py`. Repair per §2.4 — chmod will *not* fix this
`Unknown entry type 'ti-board-config'` `No module named 'yamllint'`	`sudo apt install yamllint` (note: **not** `python3-yamllint`)
binman traceback about `elftools` or missing module	Run `sudo apt install python3-pyelftools python3-yaml python3-jsonschema`
`ImportError: cannot import name 'QUIET_NOTFOUND' from 'libfdt'`	Harmless if you ran `binman` by hand. The Makefile puts the freshly built `pylibfdt` on `PYTHONPATH`; standalone invocation does not
`error: externally-managed-environment` from pip	PEP 668 — install the modules from apt, not pip (see §2.2)
`Configuration file ".config" not found`	The `make O=... <defconfig>` step failed or was skipped. Re-run it before the build command
No `-gp-evm` files produced	`BINMAN_INDIRS` not set, or points at the wrong `ti-linux-firmware`
Board totally silent on UART	`tiboot3.bin` misnamed, wrong silicon variant (HS vs GP), or partition not FAT/bootable
R5 SPL banner appears, then hangs	`sysfw.itb` missing from the boot partition
Hangs after R5, no A72 banner	`tispl.bin` bad — stale `ti-linux-firmware`, or wrong `BL31` / `TEE` path
`Cannot find /dev/sdX1`	Placeholder not replaced with the real node from `lsblk`

Derived from `doc/board/ti/j721e_evm.rst` and `doc/board/ti/k3.rst` in the `v2026.01-Beagle` tree, with the `j721e_evm_*` defconfigs substituted for the BeagleBone AI-64 equivalents, then verified by running the full build on Ubuntu 24.04 / Python 3.12. Sections 2.2, 2.4, 6, 7, 8 and 12 reflect corrections found during that run. Generated 2026-07-09.